

# DocuMind: An AI-Powered Document Intelligence and Summary Assistant Using Hierarchical Summarisation and Multimodal Vision OCR

Vallabhaneni Yaswanth Krishna  
B.Tech, Computer Science and Engineering (CSE)  
Email: vykrishna2006@gmail.com

**Abstract**—Generic chatbot-style document summarisation loses structural context and degrades sharply on long or scanned documents. This paper presents *DocuMind*, a full-stack AI document intelligence platform that replaces a single raw-document-to-LLM call with a structured, multi-stage pipeline: intelligent extraction routing between native PDF text parsing and multimodal vision OCR, text normalisation, paragraph-aware semantic chunking, hierarchical AI summarisation, and evidence-aware insight and quality extraction. The system is implemented as a FastAPI backend orchestrating seven specialised services and a React 18 single-page frontend that presents results through an interactive analysis dashboard. Given a PDF, PNG, JPG, or WEBP file, DocuMind returns three summary granularities (Executive, Standard, Detailed), six cited key insights, four actionable improvement suggestions, automatically detected document sections, and quantitative clarity/structure/completeness quality scores, all generated by a single structured-JSON call to Google’s Gemini large language model with an automatic multi-model fallback chain. We describe the system architecture, the eight-stage processing pipeline, the API contract, the user interface, and discuss the design trade-offs, current limitations, and directions for future work.

**Index Terms**—document intelligence, large language models, hierarchical summarisation, optical character recognition, FastAPI, React, Gemini, semantic chunking, natural language processing

## I. INTRODUCTION

The ability to quickly understand the content of a long document – a research paper, contract, invoice, or report – is a recurring need across academic, business, and personal workflows. The dominant approach offered by most consumer AI tools today is to paste or upload a document directly into a chatbot and ask for a summary. While convenient, this approach has three structural weaknesses. First, very long documents exceed practical context budgets or get silently truncated, causing the model to summarise only the beginning of a file. Second, a single undifferentiated prompt cannot easily produce multiple summary granularities, structured citations, or quantitative quality metrics in a form that a user interface can render reliably. Third, scanned documents and photographed pages require an entirely different extraction path than born-digital PDFs, which naive “upload-and-ask” tools frequently do not handle.

DocuMind is designed to address these weaknesses by treating document understanding as an *engineering pipeline*

rather than a single model call. The system performs intelligent extraction routing, semantic chunking, and hierarchical summarisation, and asks the underlying language model to return a strictly structured JSON payload that is validated against typed schemas before it ever reaches the user interface. This paper documents the resulting system: its architecture, its processing pipeline, its API, its user-facing dashboard, and the engineering decisions behind it.

The remainder of this paper is organised as follows. Section II states the problem and reviews the motivation for a pipeline-based approach. Section III presents the overall system architecture. Section IV details the eight-stage document processing pipeline. Section V describes the hierarchical AI summarisation strategy. Section VI documents the REST API contract. Section VII describes the frontend user interface. Section VIII discusses implementation and technology choices. Section IX covers error handling and reliability. Section X discusses limitations, and Section XI concludes with directions for future work.

## II. PROBLEM STATEMENT AND MOTIVATION

Existing document-analysis tools tend to fall into two categories. The first is a thin wrapper around a single LLM call, where the document is passed directly to the model with a summarisation prompt. This approach loses document structure, cannot cite specific pages or sections with confidence, and produces inconsistent quality on documents that are scanned images rather than extractable text. The second category is enterprise document-intelligence software, which is often accurate but expensive, closed, and difficult to integrate into a lightweight workflow.

DocuMind is designed to demonstrate that a well-engineered, open pipeline – combining deterministic extraction and chunking with a single, carefully structured AI call – can produce a dramatically richer and more reliable result than passing a raw document to a chatbot, without requiring an enterprise subscription. The guiding design principle is: *do as much as possible deterministically (extraction, cleaning, chunking, section detection) and ask the AI model to do only what only a language model can do* (semantic synthesis, insight extraction, and qualitative judgement), returned in a machine-checkable structured format.

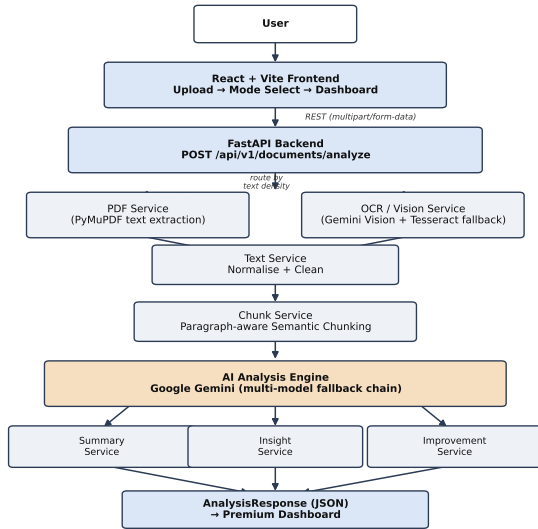


Fig. 1: DocuMind system architecture. The frontend communicates with a single FastAPI endpoint, which orchestrates extraction, normalisation, chunking, and a unified AI analysis call before returning a strictly typed JSON response.

### III. SYSTEM ARCHITECTURE

DocuMind follows a classic three-tier architecture: a React single-page application (SPA) frontend, a FastAPI backend that exposes a single primary analysis endpoint, and an external AI provider (Google Gemini) invoked exclusively from the server side so that no API key is ever exposed to the browser. Fig. 1 shows the full request flow.

The backend is organised as a set of narrowly scoped services, each with a single responsibility, coordinated by a `DocumentService` orchestrator:

- **PDFService** – extracts text from born-digital PDFs using `PyMuPDF` (`fitz`), and estimates a text-density confidence score used to decide whether OCR is required.
- **OCRSERVICE** – performs multimodal extraction on images and scanned PDF pages. It calls Gemini Vision as the primary extraction engine (rendering each PDF page to a 200 DPI bitmap when the source is a scanned PDF) and falls back to a local Tesseract installation if the vision call is unavailable.
- **TextService** – normalises raw extracted text: stripping control characters, collapsing redundant whitespace, removing page-number artefacts, and performing lightweight language detection.
- **ChunkService** – implements paragraph-aware semantic chunking with configurable chunk size and overlap, and a heading-detection heuristic used to build a document section outline.
- **SummaryService, InsightService, ImprovementService** – specialised, independently callable services that also serve as a deterministic heuristic fallback path if the unified AI call fails.

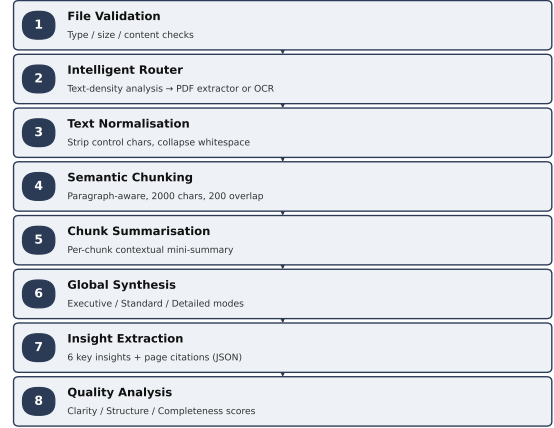


Fig. 2: The eight-stage DocuMind document processing pipeline, from raw file upload to quantitative quality analysis.

- **GeminiAIClient** (`ai_client.py`) – a centralised, resilient wrapper around the Google GenAI SDK that issues the unified structured-JSON analysis call, automatically retrying across an ordered list of fallback models on transient errors.

This separation of concerns keeps each unit independently testable and allows the system to gracefully degrade: if the single unified AI call fails or returns malformed JSON, `DocumentService` automatically falls back to invoking the three specialised services independently, each of which has its own heuristic (non-AI) fallback implementation.

### IV. DOCUMENT PROCESSING PIPELINE

Every uploaded file passes through the same eight-stage pipeline before a response is returned to the client, shown in Fig. 2.

#### A. File Validation

Uploaded files are checked against an allow-list of extensions (`pdf`, `png`, `jpg`, `jpeg`, `webp`) and a configurable maximum size (20 MB by default) before any processing begins, returning an HTTP 422 response immediately on failure.

#### B. Intelligent Extraction Routing

For PDF input, the system first attempts native text extraction with `PyMuPDF`. The average character count per page is compared against a minimum-density threshold (30 characters/page); documents below this threshold are classified as scanned and re-routed to the OCR/vision pipeline, which rasterises each page at 200 DPI and submits it to Gemini Vision for transcription. Image files are always routed directly to the vision OCR path.

### C. Text Normalisation

The `TextService` removes non-printable control characters, collapses repeated whitespace and blank lines, and strips common page-artifact patterns (headers, footers, page numbers) before word-count and heuristic language detection are computed.

### D. Semantic Chunking

Rather than chunking by a fixed character window (which risks splitting sentences mid-way), `ChunkService` splits the normalised text on paragraph boundaries and accumulates paragraphs into chunks of approximately 2,000 characters, carrying up to 200 characters of trailing context forward into the next chunk to preserve continuity across chunk boundaries. In parallel, a regular-expression heuristic scans line-level text for heading-like patterns to build a navigable document section outline (used by the frontend’s Document Explorer).

### E. Chunk Summarisation and Global Synthesis

Individually summarised chunk-level context is available as a deterministic fallback path (`SummaryService`), but in the default, healthy-path execution DocuMind performs *hierarchical summarisation in a single optimised call*: a representative sample of the normalised document (the first 6,000 characters) is sent to Gemini with an explicit JSON schema requesting simultaneous Executive, Standard, and Detailed summaries, described further in Section V.

### F. Insight Extraction and Quality Analysis

The same unified AI call returns exactly six key insights (each under 25 words, with an optional source-page citation field) and exactly four categorised, actionable improvement suggestions (categories: Clarity, Structure, Completeness, Recommendation), alongside four integer quality sub-scores (Clarity, Structure, Completeness, Overall) on a 0–100 scale. All numeric scores are clamped server-side to a sane range (10–100) before being returned, defending against malformed model output.

## V. HIERARCHICAL AI SUMMARISATION STRATEGY

The central design decision in DocuMind is to avoid sending an entire long document to the language model in a single unstructured prompt. Instead, the system uses what we term *hierarchical summarisation*: extraction and chunking are performed deterministically, and the language model is asked – in one structured call, constrained by an explicit JSON schema in the prompt – to synthesise all three summary granularities together, rather than issuing three separate round trips. Fig. 3 illustrates the conceptual flow, including the fully decomposed chunk-summarise-then-synthesise mode used as the deterministic fallback path when the unified call is unavailable.

This design has two practical benefits. First, requesting all three summary lengths from one call (rather than three independent calls) keeps the summaries mutually consistent – the Detailed summary is a strict elaboration of the same narrative

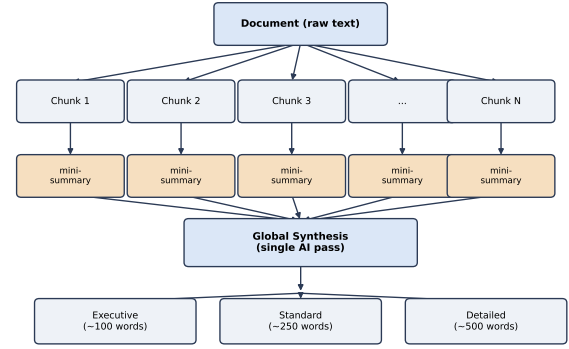


Fig. 3: Hierarchical summarisation: chunk-level mini-summaries are synthesised into three output granularities in a single global pass.

as the Executive summary, rather than an independently generated (and potentially contradictory) text. Second, constraining the model to return typed JSON (parsed and validated with Pydantic v2 on the server) means the frontend never has to parse free-form natural language to populate structured UI components such as quality-score bars or numbered insight lists.

The client-facing prompt explicitly enumerates the required JSON structure (summary object, key\_insights array, improvements array, quality object) and a short set of writing-style guidelines (e.g., “third-person prose, no bullet points inside summary values”), and the `GeminiAIClient` strips Markdown code-fences and attempts a best-effort regular-expression JSON extraction if the model’s raw output is not perfectly parseable, before falling back to the deterministic heuristic services on total failure.

## VI. API DESIGN

DocuMind exposes a deliberately small, predictable API surface. The primary endpoint is:

```
POST /api/v1/documents/analyze
```

accepting `multipart/form-data` with a required `file` field (PDF/PNG/JPG/WEBP,  $\leq 20$  MB) and an optional `summary_mode` field (`executive` | `standard` | `detailed`, defaulting to `standard`). Table I summarises the top-level fields of the resulting `AnalysisResponse` object, which is fully described by typed Pydantic schemas (`DocumentStats`, `SummaryResult`, `KeyInsight`, `ImprovementItem`, `DocumentSection`, `QualityScore`) and auto-documented via FastAPI’s interactive Swagger UI at `/api/docs`.

A secondary `GET /health` endpoint reports the operational status of the API, PDF extractor, OCR engine, and AI pipeline for uptime monitoring and load-balancer health checks. Error responses use standard HTTP status codes: 422 for invalid, oversized, or empty/unreadable input, and

TABLE I: Top-Level Fields of the AnalysisResponse Object

| Field                 | Description                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------|
| document              | Filename, inferred type, page/word/char counts, language, extraction method, confidence               |
| extraction            | Extraction method used and a 300-character raw-text preview                                           |
| summary               | Executive, Standard, and Detailed summary text                                                        |
| key_insights          | Six numbered insights with optional page citation                                                     |
| improvements_sections | Four categorised, actionable improvement items<br>Up to ten detected document sections with pre-views |
| quality               | Clarity, Structure, Completeness, Overall (0–100)                                                     |
| full_text             | Complete normalised document text                                                                     |

500 for upstream AI service failure, each with a human-readable detail message that the frontend surfaces as a toast notification with a retry option.

POST /api/v1/documents/analyze and GET /health constitute the complete server-side API surface of the deployed project: the backend intentionally exposes no other public routes, and all further application state (authentication, analysis history) is delegated to Supabase rather than to bespoke backend endpoints, keeping the FastAPI service itself stateless and easy to reason about.

## VII. USER INTERFACE

The frontend is a React 18 single-page application built with Vite and styled with Tailwind CSS and Framer Motion for animation. Beyond the core upload-and-analyse flow, the application includes Supabase-backed authentication (Login and Register pages, a ProtectedRoute wrapper, and an AuthContext provider), a Profile page, and a History page that lists a user’s previous analyses – extending the core document-intelligence pipeline into a persistent, multi-session product rather than a single-shot demo.

The central experience is the Analysis dashboard, reached after a file is uploaded on the Home page and a summary mode is selected. An animated ProcessingOverlay component displays the pipeline stages (extraction, chunking, AI analysis) as they execute, giving the user visible feedback during the 5–30 second processing window. Fig. 4 shows a reconstruction of the resulting dashboard layout as rendered in the deployed application.

The dashboard is composed of independently reusable components: DocumentStats (page count, word count, language, extraction confidence), SummaryPanel (a tabbed switcher across the three summary granularities), KeyInsights (a numbered, optionally page-cited list), ImprovementSuggestions (categorised, icon-coded actionable feedback), and DocumentViewer (a Document Explorer with section navigation over the full extracted text). Data fetching and state management for the analysis flow are encapsulated in a custom useDocumentAnalysis hook, keeping the presentational components free of networking logic.

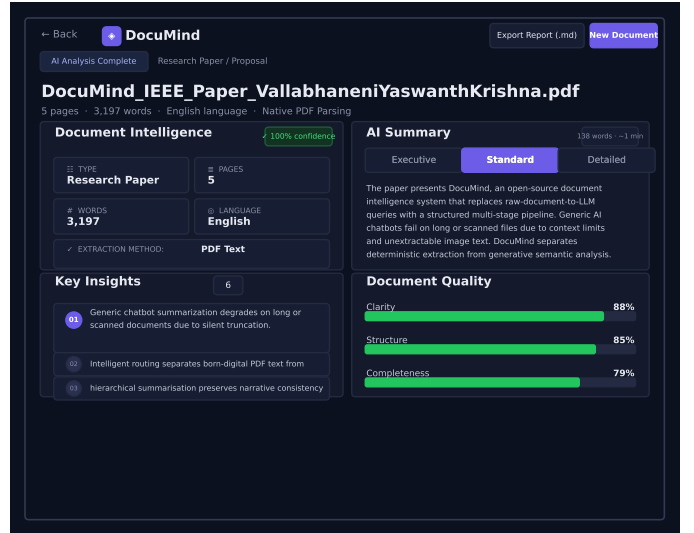


Fig. 4: Reconstruction of the live DocuMind Analysis dashboard (dark theme), matching the deployed application: header actions (Export Report, New Document), the Document Intelligence stat grid, the tabbed AI Summary panel, numbered Key Insights, and Document Quality score bars, corresponding directly to DocumentStats.jsx, SummaryPanel.jsx, KeyInsights.jsx, and the quality section of DocumentViewer.jsx.

TABLE II: DocuMind Technology Stack

| Layer              | Technology                                                             |
|--------------------|------------------------------------------------------------------------|
| Frontend           | React 18 + Vite, React Router                                          |
| Styling / Motion   | Tailwind CSS, Framer Motion                                            |
| Auth & Persistence | Supabase (auth, history storage)                                       |
| Backend            | FastAPI (Python 3.11+), Uvicorn                                        |
| Validation         | Pydantic v2 (typed request/response schemas)                           |
| PDF Extraction     | PyMuPDF (fitz)                                                         |
| OCR / Vision       | Google Gemini Vision, Tesseract (pytesseract) fallback                 |
| AI Reasoning       | Google Gemini (structured JSON generation, multi-model fallback chain) |
| Deployment         | Vercel (frontend), Render (backend)                                    |

## VIII. IMPLEMENTATION AND TECHNOLOGY STACK

Table II summarises the technology choices across both tiers of the application.

The backend’s GeminiAIClient maintains an ordered list of candidate models (a configured primary model followed by progressively lighter-weight fallback models) and automatically retries a failed call against the next candidate on transient errors such as HTTP 429/503/404 responses, with a short back-off delay between attempts. This gives the pipeline resilience against transient quota exhaustion or regional model unavailability without requiring any client-visible retry logic. Configuration – API keys, allowed CORS origins, file-size limits, chunk size/overlap, and per-mode summary word targets – is centralised in a single Pydantic Settings object loaded from environment variables, keeping deployment configuration outside of source code.

TABLE III: Error Handling Scenarios

| Scenario                | Response                                                   |
|-------------------------|------------------------------------------------------------|
| Unsupported file type   | HTTP 422 – “Please upload PDF, PNG or JPG”                 |
| File exceeds size limit | HTTP 422 – “File exceeds 20 MB limit”                      |
| Empty / blank scan      | HTTP 422 – “No readable text detected”                     |
| AI service failure      | HTTP 500, deterministic heuristic fallback attempted first |
| Network error (client)  | Toast notification with retry option                       |

## IX. ERROR HANDLING AND RELIABILITY

DocuMind applies defensive handling at every pipeline boundary. Unsupported file types and oversized uploads are rejected before any extraction is attempted. Documents that yield no readable text after extraction (blank pages, heavily corrupted scans) raise an explicit, user-readable error rather than silently returning an empty analysis. Table III summarises the primary error scenarios and their handling.

Crucially, an AI-layer failure does not necessarily surface as an error to the user at all: because `DocumentService._analyze_ai` wraps the unified Gemini call in a broad exception handler, a failure at that stage transparently falls back to the three independent heuristic services (extractive, non-AI summarisation and rule-based insight/quality estimation), so that the pipeline degrades gracefully rather than failing outright whenever the AI provider is briefly unavailable.

## X. LIMITATIONS

The current implementation has several known limitations. OCR accuracy is dependent on scan quality, with 300 DPI recommended as a practical minimum for the local Tesseract fallback path. Very large documents (100+ pages) may take 30–60 seconds to process end-to-end due to sequential per-page OCR when the vision path is used on scanned input. Language detection is heuristic rather than model-based; non-English documents are processed through the same pipeline but may yield lower-quality summaries, since the unified prompt is authored in English. Section detection relies on a regular-expression heading heuristic and can under- or over-segment documents with unconventional formatting. Finally, because the unified AI call samples only the first 6,000 characters of the normalised document as direct context (relying on the earlier deterministic chunking/summarisation path only as a fallback), summaries of extremely long documents are weighted toward their opening sections.

## XI. CONCLUSION AND FUTURE WORK

DocuMind demonstrates that a deliberately engineered document-intelligence pipeline – combining deterministic extraction routing, text normalisation, paragraph-aware chunking, and a single structured-JSON AI call with automatic model fallback and heuristic degradation – can produce a substantially richer, more reliable, and more UI-ready result than treating document summarisation as a single unstructured chatbot prompt. The system’s service-oriented backend

and component-based frontend keep each processing stage independently testable, and its graceful-degradation design means the user-facing experience remains functional even under partial AI-provider failure.

Planned future improvements include genuine multi-language support via automatic language detection paired with locale-specific prompting, vector-database integration to support conversational document question-answering over previously analysed files, batch (multi-document) processing, export of generated summaries to PDF/DOCX, and expanded analytics on top of the existing Supabase-backed user history feature.

## ACKNOWLEDGMENT

The author thanks the open-source maintainers of FastAPI, React, PyMuPDF, and Tesseract OCR, whose libraries form the technical foundation of this project.

## REFERENCES

- [1] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, 2024.
- [2] Meta Open Source, “React – A JavaScript library for building user interfaces,” <https://react.dev/>, 2024.
- [3] Artifex Software, “PyMuPDF Documentation,” <https://pymupdf.readthedocs.io/>, 2024.
- [4] R. Smith, “An Overview of the Tesseract OCR Engine,” in *Proc. Ninth Int. Conf. on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [5] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv:2312.11805, 2023.
- [6] S. Colvin et al., “Pydantic,” <https://docs.pydantic.dev/>, 2024.
- [7] A. See, P. J. Liu, and C. D. Manning, “Get To The Point: Summarization with Pointer-Generator Networks,” in *Proc. 55th Annual Meeting of the ACL*, 2017, pp. 1073–1083.